

DV1633 Scripting and Other Languages

Course Project (4.5 HP, G-U)

March 2th, 2026.

Deadline 1	May 29th, 2026 23:59
Deadline 2	August 21th, 2026 23:59
Deadline 3	January 1th, 2027 23:59
Submission	ZIP or RAR file uploaded to Canvas

Developed by Patrik Svensson and Peng Wang, 2026.

Description

This project offers students an opportunity to apply the different concepts and techniques explored throughout the DV1633 Scripting and Other Languages course, into the development of a game editing tool using the *Lua* scripting language.

Specifically, the project evaluates the capabilities of the student to implement game logic and/or tool manipulation scripts using Lua, analyze and motivate how and when Lua should be used in combination with other programming languages, and establish co-routines between Lua and an existing video game, to modify or edit a specific system from it.

Instructions

For the development of this course project, students are tasked with implementing two different applications: a *video game*, and a *game editing tool*. The main goal of the project is to modify one specific system of the video game, together with any element associated with it, by using the game editing tool during runtime. The students can determine which video game system will be altered by

the game editing tool. The following subsections expose in greater detail the implementation parameters for the development of these two applications.

Video Game

The parameters for implementing the video game are:

- **Type of game:** students are allowed to implement any kind of 2D or 3D game for this project, as long as it doesn't portray any content that surpasses the guidelines established for the PEGI 12 rating. It is **strongly recommended**, however, to build a **2D or 2.5D game** to minimize the implementation complexity.
- **Scope:** the scope of the video game should be small, and must be realistically possible to implement it within the time allocated for the project (approx. four weeks).
- **Implementation language:** the video game must be implemented using C/C++ and Lua.
- **Pre-existing tools:** students are allowed to select and use any kind of pre-existing frameworks or development tools (e.g. libraries, APIs, etc.) that may reduce the implementation time of the video game. Despite this, the video game must be designed and developed by the students themselves. It is **strongly recommended** to use **Raylib** for the implementation of the video game.
- **ECS:** the game must feature an *entity component system (ECS)*, to register and manage the different elements used among its other systems. **Entities** must be created using **Lua exclusively**, while Components and Systems should be created using C/C++. The ECS must be able to be exposed to the game editing tool. For the implementation of the ECS, the **EnTT library** is **strongly recommended**.
 - The use of *proxy functions* (functions that are declared using Lua, but that are called from C/C++ processes) are not allowed for the creation of Entities in the ECS.

Game Editing Tool

The parameters for implementing the game editing tool are:

- **Embedded system:** the game editing tool must be embedded within the video game.
- **Access and edit the ECS:** the game editing tool must be able to access and modify the information from the video game ECS. Specifically, the game editing tool **must allow Lua to:**

1. Create entities.
 2. Create behaviors for entities called from C/C++, using co-routines.
- **Game states:** the game editing tool must be able to modify the state of the game, by manipulating any specific game system. The modified game state must be saved to an external file and loaded into the video game, using a self-defined format.

Project Idea Pitch

The course project presents students with a lot of freedom to design and implement the video game and game editing tool. Therefore, to assess the viability of the project idea, a pitch must be carried out by the students before its implementation.

In preparation for the project idea pitch, students must write a report in which they specify the kind of video game, the planned functionality for it, how each programming language will be used, and what system will be modified through the use of the game editing tool. An example of a written report for the project idea pitch is available in the *Course Project* module in Canvas. Make sure to read it!

A project idea will only be allowed for submission if it has been approved during the pitch.

Oral Examination

After completing the course project submission, students will be asked to present and defend the video game and the game editing tool they have implemented, in an oral examination. There, examiners will assess that students developed a comprehensive understanding of the technical concepts of Lua, and its use in combination with other programming languages.

Assesment

The course project submission will be graded G upon successfully complying with all the implementation parameters mentioned in the Instructions section, and after successfully present and defend their submission in the oral examination. A U grade will be given if the video game or the game editing tool are incomplete or have major compilation/runtime errors (e.g. game not being able to initialize or close, game crashing during gameplay, gameplay systems not working, etc.), or if the oral examination shows that students do not understand the technical aspects of their submission. Lastly, a Ux grade will be given if the

video game or game editing tool require minor adjustments.

The development of this course project must take place either individually or in groups of two students. However, we strongly recommend that you work in groups. Also, **any found case of plagiarism will be reported and will have disciplinary actions taken against the students involved.**

The submission for this project **must**:

- Include the video game and the game editing tool.
- Include all necessary files/libraries/binaries to correctly build and run your submission on a computer different than yours.
- Present a written document in which student names, building instructions, and usage instructions are provided to the examiners, since they are not allowed to troubleshoot any submission.
- Be compatible with Microsoft Windows.

Submit all this information within a **.zip or .rar file**.

Happy Coding!